Parallel & Distributed Computing Lab

IMSciences

Lab Instructor: Saim Sanan

Week 1, Lab 2

Flynn's Taxonomy and POSIX Threads

---

1 Ideas

Four ways a machine can issue instructions and touch data:

- **SISD** — Single Instruction, Single Data. Classic sequential CPU.
- **SIMD** — Single Instruction, Multiple Data. One instruction, many data (GPU, vector units).
- **MISD** — Multiple Instruction, Single Data. Rare.
- **MIMD** — Multiple Instruction, Multiple Data. Many cores, each can run a different instruction on different data. A typical lab PC or VM is MIMD.

> **SPMD is not a Flynn class.** Flynn describes the *machine*. SPMD is how the *code* is written: same C file, many threads or processes. MPI programs are often SPMD on MIMD hardware.

2 Pthreads

A process starts with one main thread. `pthread_create` starts extra workers in the *same* process. They share memory.

- `pthread_create` — start a thread.
- `pthread_join` — wait until that thread finishes. If `main` returns first, the process dies and the other threads stop.
- Compile with `-pthread`. Without it, the linker cannot find `pthread_create`.

Print order is **not** 1 then 2 then 3 then 4 every time. The OS scheduler picks who runs. Run the program three times. If the order changes, that is the scheduler, not a broken `printf`.

3 Commands

Terminal

```
nano hello_threads.c
gcc -pthread hello_threads.c -o hello_threads
./hello_threads
./hello_threads
./hello_threads
```

nano: Ctrl+O, Enter (save), Ctrl+X (quit).
Forgot `-pthread` → undefined reference to `pthread_create`.
Forgot `./` → command not found.

4 Program

Four threads. Each gets its own `id` in an array. Do not pass `&i` from the loop — that is one variable four threads share.

C

```c
#include <stdio.h>
#include <pthread.h>

void *hello(void *arg) {
    int id = *(int *)arg;
    printf("Hello from thread %d\n", id);
    return NULL;
}

int main(void) {
    pthread_t t[4];
    int id[4];
    int i;

    for (i = 0; i < 4; i++) {
        id[i] = i + 1;
        pthread_create(&t[i], NULL, hello, &id[i]);
    }

    for (i = 0; i < 4; i++) {
        pthread_join(t[i], NULL);
    }

    printf("main done\n");
    return 0;
}
```

Change 4 to 8 in both arrays and both loops, compile again, run. Eight Hellos, then `main done`.